Projeto final de automação IoT

Etapa 2: projeto técnico e arquitetura da solução

Monitor IoT de temperatura para servidor
Engenharia de Software · Internet das Coisas
Católica de Santa Catarina, Centro Universitário
Professor: Msc. Rodrigo Hagstrom

Este documento define como o protótipo será construído: blocos, componentes, pinos, ligações, mensagens e lógica de controle. O circuito já foi montado e testado no simulador Wokwi (Anexo A) e será montado na bancada na Etapa 3. As mudanças em relação à proposta da Etapa 1 estão na seção 11.

Integrantes (Grupo 2): Bruno Ruan dos Santos, Bernardo Wehmuth, João Vítor Dal-Ri, João Vítor Círico, Nicolas Soares Oliveira e Vinícius Bueno de Oliveira.

Data de entrega: 06/11/2026

1. Diagrama de blocos

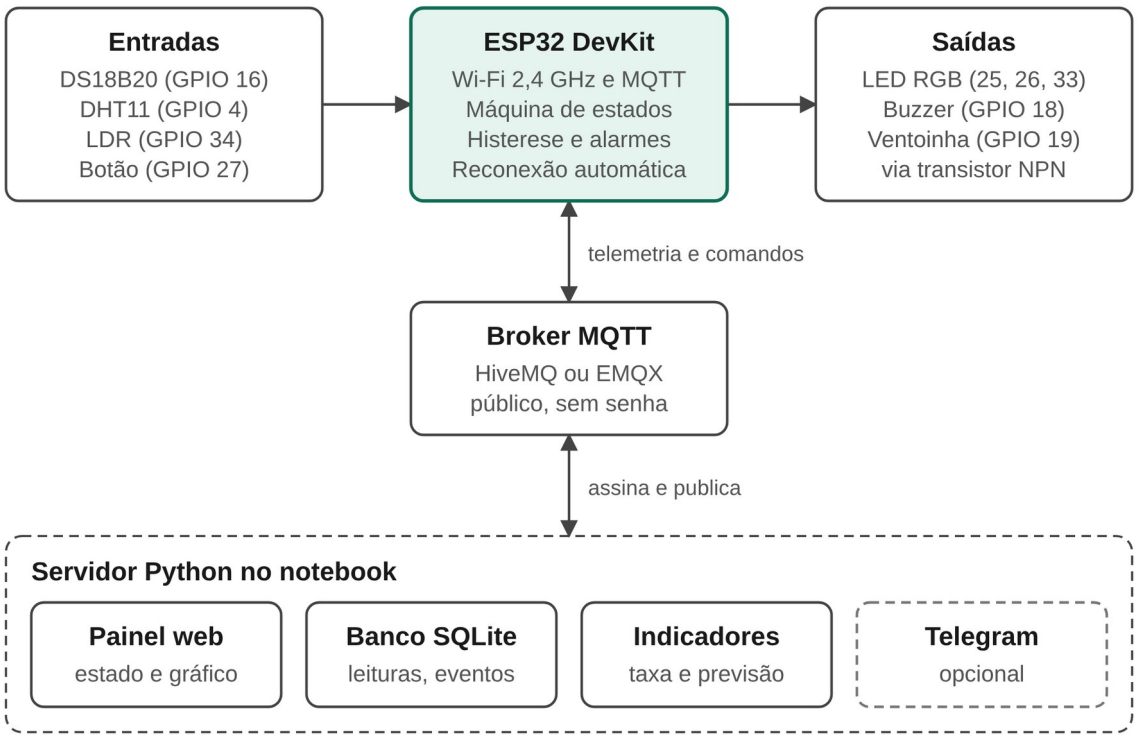


Figura 1. Diagrama de blocos da solução: entradas, ESP32, saídas, broker MQTT e servidor Python.

O ESP32 executa a lógica de controle sozinho: lê as entradas, define o estado e aciona as saídas. A rede serve para enviar dados e receber comandos, e o controle local continua funcionando sem ela. O servidor Python, no notebook, assina os tópicos do broker, grava os dados, exibe o painel e calcula os indicadores. No modo automático, ele não participa da decisão de ligar a ventoinha.

2. Especificação de sensores, atuadores e módulos

Componente	Função	Interface	Alimentação	Especificações e observações
ESP32 DevKit	Controle, Wi-Fi 2,4 GHz e MQTT	GPIOs de 3,3 V	USB, 5 V	Os pinos 34 a 39 são somente entrada. O ADC2 não funciona com o Wi-Fi ligado.
DS18B20	Temperatura principal	1-Wire, GPIO 16	3,0 a 5,5 V (usa 3V3)	Erro de ±0,5 °C na faixa de -10 °C a +85 °C. Resolução configurável de 9 a 12 bits. Pull-up de 4,7 kΩ.
DHT11	Umidade e temperatura de apoio	Fio único, GPIO 4	3V3 (5V se a leitura falhar)	Erro de ±2 °C e ±5% de umidade relativa; faixa de 0 a 50 °C e de 20 a 80% UR, segundo o material do kit. Pull-up de 10 kΩ ao 3V3. O kit indica de 3,5 a 5,5 V, mas o sensor funcionou com 3,3 V num teste anterior no laboratório.
LDR	Detecta a tampa aberta	Analógica, GPIO 34 (ADC1)	3V3	Divisor de tensão: resistor de 10 kΩ entre o 3V3 e o GPIO, LDR entre o GPIO e o GND. Com mais luz, a leitura cai. O limiar será calibrado na bancada.
Botão	Silencia o buzzer	Digital, GPIO 27	Sem alimentação	Usa INPUT_PULLUP. O outro terminal vai ao GND.
LED RGB	Estado do sistema	3 saídas digitais, GPIO 25, 26 e 33	Pelos GPIOs (3,3 V)	Resistor de 300 Ω por cor. O LED do kit é de cátodo comum, segundo o material do kit: o pino mais comprido vai ao GND.
Buzzer	Alarme sonoro	Saída digital, GPIO 18	Pelo GPIO	Passivo, de 5 V e piezoelétrico, segundo o material do kit. Toca com `tone()` e liga direto ao GPIO.
Ventoinha de 5 V	Resfriamento do gabinete	Saída digital via transistor, GPIO 19	5 V (VIN)	Transistor NPN (BC337 ou 2N2222), resistor de 300 Ω na base e diodo 1N4007 em paralelo. A corrente será confirmada na peça comprada.

3. Alimentação e cuidados elétricos

- O ESP32 é alimentado pela USB (5 V). A lógica dos GPIOs é de 3,3 V, e nenhum pino recebe 5 V.
- O pino 3V3 alimenta o DS18B20, o DHT11, os pull-ups dos dois e o divisor do LDR. O DS18B20 aceita de 3,0 a 5,5 V.
- O material do kit indica de 3,5 a 5,5 V para o DHT11, mas ele funcionou com 3,3 V num teste anterior no laboratório. Por isso ele começa ligado ao 3V3, com o pull-up também no 3V3. Se a leitura falhar, passa para o pino 5V (VIN), com o pull-up mantido no 3V3 para que o sinal de dados não ultrapasse 3,3 V. Nesse caso, a tensão no pino de dados será medida com multímetro antes de ligá-lo ao ESP32.
- A ventoinha não liga direto no GPIO. O GPIO 19 aciona a base de um transistor NPN por um resistor de 300 Ω. O emissor vai ao GND e o coletor ao terminal negativo da ventoinha, cujo positivo vai ao 5 V. O diodo 1N4007, em paralelo com a ventoinha e com o catodo no 5 V, absorve o pico de tensão quando ela desliga.
- A corrente de base é de cerca de 8,7 mA (3,3 V menos 0,7 V, dividido por 300 Ω). Deve bastar para uma ventoinha pequena, e a corrente real será confirmada na etiqueta da peça.

- Se o ESP32 reiniciar quando a ventoinha ligar, um capacitor eletrolítico de 100 µF entre 5 V e GND, perto da ventoinha, estabiliza a alimentação. Em último caso, a ventoinha passa para uma fonte própria com GND comum.
- O GND é comum a ESP32, sensores, transistor e ventoinha.
- O LDR fica entre o GPIO 34, que pertence ao ADC1, e o GND, com o resistor de 10 kΩ entre o 3V3 e o GPIO. Os pinos 0, 2, 12 e 15 não são usados, porque influenciam a inicialização do ESP32.
- Com 3,3 V, o verde e o azul do LED RGB podem ficar fracos com resistores de 300 Ω. Se isso acontecer, os resistores serão reduzidos ou o LED será trocado. O buzzer do kit é passivo e piezoelétrico, então consome pouco e liga direto ao GPIO. Ele toca com `tone()`, e o volume pode ser menor por usar 3,3 V.

4. Tabela de GPIOs

Pino	Função	Tipo	Componente	Observação
GPIO 4	Dado do DHT11	Entrada digital	DHT11	Pull-up de 10 kΩ ao 3V3
GPIO 16	Dado do DS18B20	1-Wire	DS18B20	Pull-up de 4,7 kΩ ao 3V3
GPIO 18	Alarme sonoro	Saída digital	Buzzer	Passivo de 5 V: usar `tone()`
GPIO 19	Aciona a ventoinha	Saída digital	Base do transistor	Resistor de 300 Ω em série
GPIO 25	Vermelho do LED RGB	Saída digital	LED RGB	Resistor de 300 Ω
GPIO 26	Verde do LED RGB	Saída digital	LED RGB	Resistor de 300 Ω
GPIO 27	Silenciar o alarme	Entrada com pull-up interno	Botão	Outro terminal no GND
GPIO 33	Azul do LED RGB	Saída digital	LED RGB	Resistor de 300 Ω. LED de cátodo comum: nível alto acende.
GPIO 34	Leitura do LDR	Entrada analógica (ADC1)	LDR	Somente entrada. 10 kΩ no 3V3 e LDR no GND
3V3	Alimentação	Energia	DS18B20, DHT11, pull-ups e divisor do LDR	Saída do regulador do ESP32
5V (VIN)	Alimentação	Energia	Ventoinha (e o DHT11, se precisar)	Vem da USB
GND	Referência comum	Energia	Todos os componentes	Comum a todo o circuito

5. Esquema de ligação

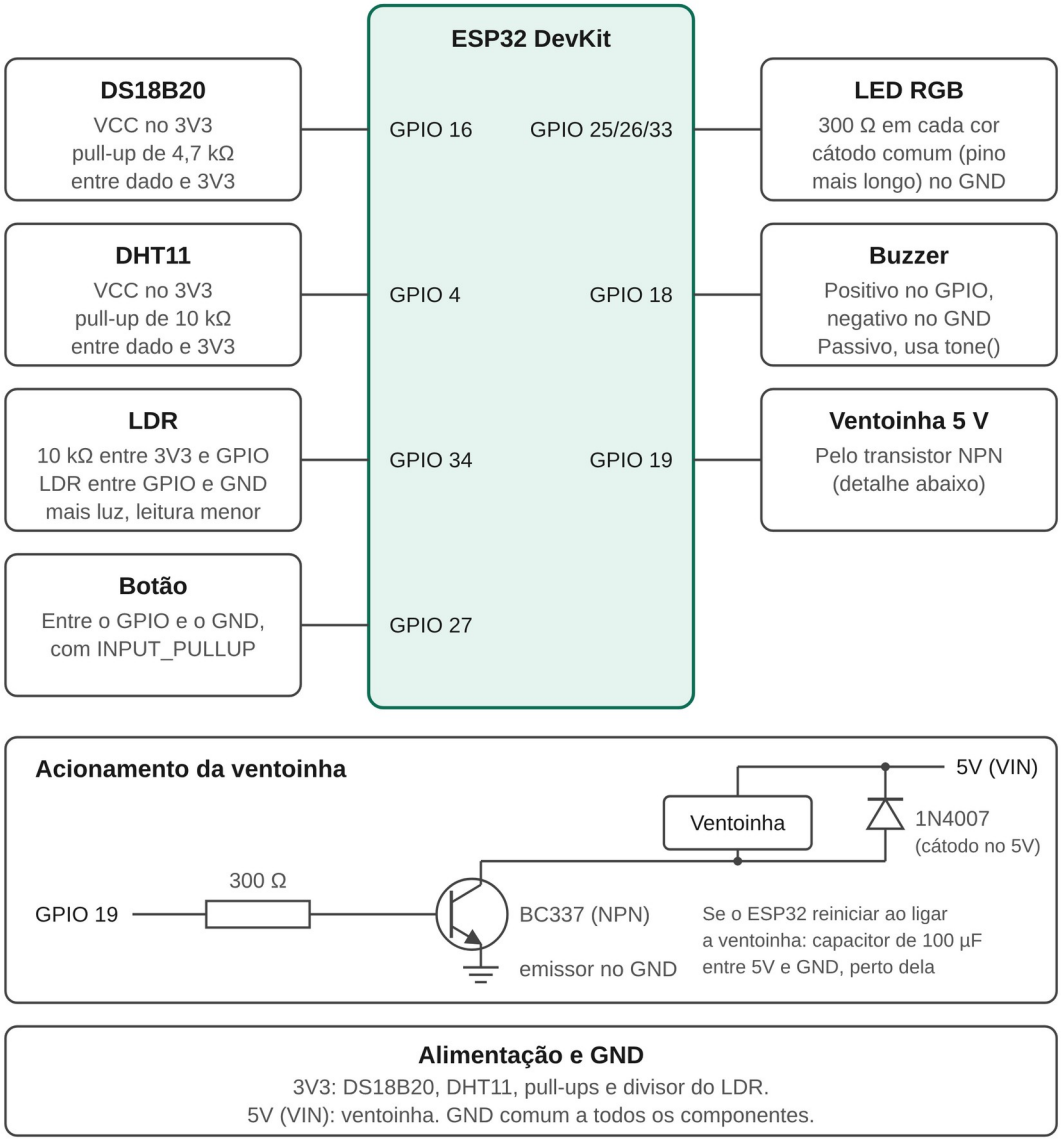


Figura 2. Ligações dos componentes ao ESP32, com os GPIOs usados e o acionamento da ventoinha.

A figura mostra a ligação de cada componente ao ESP32 e, no quadro de baixo, o circuito que aciona a ventoinha pelo transistor. O circuito montado no Wokwi, com o link do projeto, está no Anexo A (seção 13). No simulador, a ventoinha é representada por um LED.

6. Conectividade, protocolo e plataforma

A conexão usa Wi-Fi de 2,4 GHz e o protocolo MQTT sobre TCP, na porta 1883, com a biblioteca PubSubClient.

O broker será público, na nuvem: HiveMQ (`broker.hivemq.com`), com o EMQX (`broker.emqx.io`) como alternativa. O Wokwi roda na nuvem e não alcança um broker instalado no notebook, e um broker público atende ao simulador e à bancada com a mesma configuração.

A plataforma é um servidor escrito em Python, executado no notebook. Ele assina os tópicos do broker com a biblioteca paho-mqtt, grava cada mensagem no banco SQLite, publica os comandos e oferece uma API REST (FastAPI) que alimenta um painel web em HTML e JavaScript. O servidor registra a data e a hora de cada mensagem ao recebê-la, e o ESP32 envia o tempo de funcionamento (uptime).

Todo o código fica em um repositório no GitHub, dividido em firmware, servidor, painel e documentação:

```
firmware/   código do ESP32 (Arduino, C++)
server/     servidor Python, banco SQLite e testes
web/        painel web (HTML, CSS e JavaScript)
wokwi/      diagram.json e bibliotecas do simulador
docs/       documentos das etapas e diagramas
```

Se o Wi-Fi ou o broker caírem, o ESP32 tenta reconectar a cada 5 segundos sem interromper o controle local da ventoinha, do LED e do buzzer.

O broker público não tem autenticação, então qualquer pessoa que conheça os tópicos pode ler ou publicar neles. Para reduzir o problema, os tópicos levam um prefixo único da equipe e a telemetria não contém dado sensível. O servidor e o firmware validam toda mensagem recebida, o firmware só aceita limites dentro de uma faixa segura, e a ventoinha fica ligada nos estados crítico e de falha mesmo no modo manual. Um broker com usuário, senha e TLS fica como evolução possível, e a limitação será registrada no dossiê final.

7. Mensagens e comandos

Todos os tópicos começam por `thermomon/g2-axr2adzj/rack01/`. O código g2-axr2adzj junta o número do grupo a um sufixo aleatório, para não colidir com tópicos de outras pessoas no broker público. A tabela mostra o final de cada tópico. Os nomes técnicos (tópicos, campos e tabelas) estão em inglês, como o restante do código, e a explicação continua em português.

Tópico	Direção	Conteúdo	Observação
`telemetry`	ESP32 → broker	JSON, com os campos abaixo	A cada 5 s, com QoS 0.
`status`	ESP32 → broker	`online` ou `offline`	Mensagem retida, com o Last Will do MQTT (reforço).
`cmd/mode`	Servidor → ESP32	`auto` ou `manual`	Alterna o modo de operação.
`cmd/fan`	Servidor → ESP32	`on` ou `off`	Só tem efeito no modo manual e nunca desliga a ventoinha em crítico ou falha.
`cmd/limits`	Servidor → ESP32	`{"warning":27,"critical":32}`	Só aceita valores entre 20 e 50 °C, com o crítico pelo menos 3 °C acima do atenção. A histerese de 2 °C se mantém.
`cmd/silence`	Servidor → ESP32	`1`	Silencia o buzzer do estado atual.

Exemplo de mensagem de telemetria:

```
{"device":"esp32-
rack01","uptime_s":1234,"temp_c":24.31,"dht_temp_c":24.0,"humidity_pct":41,"lid_open":false,"
fan_on":true,"state":"normal","mode":"auto"}
```

Campo	Tipo	Descrição
`device`	texto	Identificação do dispositivo, por exemplo `esp32-rack01`.
`uptime_s`	inteiro	Segundos desde a partida do ESP32.
`temp_c`	número	Temperatura do DS18B20, em °C. Vai null se a leitura falhar.
`dht_temp_c`	número	Temperatura do DHT11, em °C. Vai null se a leitura falhar.
`humidity_pct`	inteiro	Umidade relativa do DHT11, em %. Vai null se a leitura falhar.
`lid_open`	booleano	Verdadeiro quando a leitura do LDR cai abaixo do limiar (tampa aberta).
`fan_on`	booleano	Verdadeiro quando a ventoinha está ligada.
`state`	texto	`normal`, `warning`, `critical` ou `fault`.
`mode`	texto	`auto` ou `manual`.

Ao reiniciar, o ESP32 volta aos limites padrão (27 e 32 °C).

8. Banco de dados

O histórico fica em um banco SQLite, um arquivo único que não precisa de servidor de banco. A tabela `readings` guarda cada mensagem de telemetria, e a tabela `events` guarda os acontecimentos pontuais, como mudança de estado e comandos enviados. As colunas repetem os campos do JSON da seção 7, para que a gravação seja direta.

Tabela `readings`:

Coluna	Tipo	Descrição
`id`	INTEGER	Chave primária, gerada pelo banco.
`device`	TEXT	Identificação do ESP32.
`received_at`	TEXT	Data e hora de chegada ao servidor, em UTC (ISO 8601).
`uptime_s`	INTEGER	Segundos desde a partida do ESP32.
`temp_c`	REAL	Temperatura do DS18B20, em °C. Fica vazia se a leitura falhar.
`dht_temp_c`	REAL	Temperatura do DHT11, em °C. Fica vazia se a leitura falhar.
`humidity_pct`	INTEGER	Umidade relativa do DHT11, em %. Fica vazia se a leitura falhar.
`lid_open`	INTEGER	0 ou 1.
`fan_on`	INTEGER	0 ou 1.
`state`	TEXT	`normal`, `warning`, `critical` ou `fault`.
`mode`	TEXT	`auto` ou `manual`.

Tabela `events`:

Coluna	Tipo	Descrição
`id`	INTEGER	Chave primária, gerada pelo banco.
`device`	TEXT	Identificação do ESP32.
`occurred_at`	TEXT	Data e hora do evento, em UTC (ISO 8601).
`kind`	TEXT	`state_change`, `sensor_fault`, `command`, `online` ou `offline`.
`detail`	TEXT	Texto livre ou JSON com o valor anterior e o novo.

9. Fluxo lógico: condição, decisão e ação

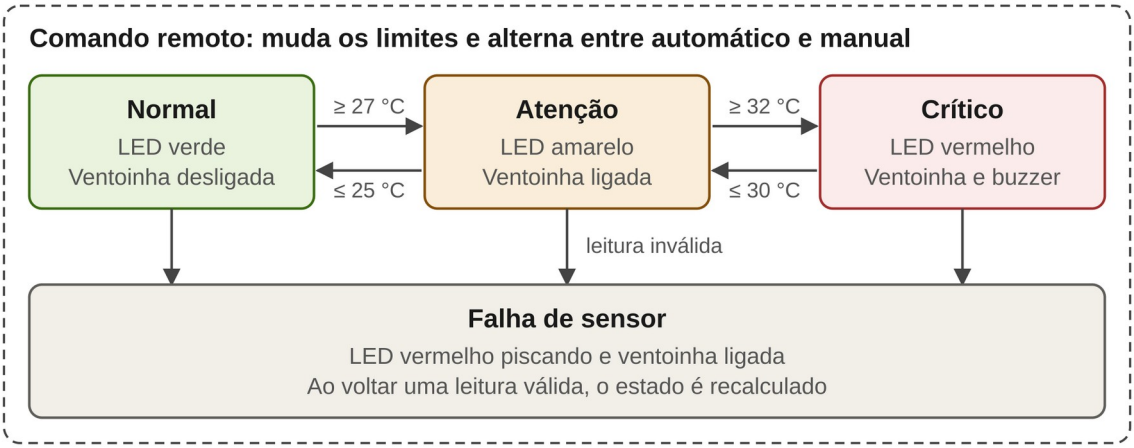


Figura 3. Estados do sistema, limites de transição e estado de falha de sensor.

A cada ciclo, o firmware lê os sensores, define o estado e aplica a ação correspondente. Sobe-se de estado ao atingir 27 °C ou 32 °C, e só se desce depois que a temperatura cai para 25 °C ou 30 °C. Essa diferença de 2 °C é a histerese. Se a temperatura já estiver em 32 °C ou mais, por exemplo na partida, o estado vai direto para crítico.

Algumas leituras inválidas seguidas do DS18B20, por sensor desconectado ou sem resposta, levam ao estado de falha. Nele, a ventoinha fica ligada por segurança e o LED vermelho pisca. Quando uma leitura válida volta, o estado é recalculado.

No modo manual, a ventoinha obedece ao comando remoto, exceto nos estados crítico e de falha, em que ela fica ligada por segurança. O LED e o buzzer continuam seguindo o estado, para que um alarme não seja desligado por engano. O botão silencia o buzzer do estado atual. A tampa aberta, detectada pelo LDR, é registrada e enviada na telemetria.

10. Análises planejadas sobre os dados

O servidor calcula os indicadores abaixo em um módulo próprio, a partir das leituras gravadas no banco, e os entrega ao painel pela API.

1. Taxa de variação da temperatura, em °C por minuto, sobre uma média móvel para reduzir o ruído.
2. Previsão do tempo até o limite crítico, por regressão linear sobre os últimos 10 minutos (cerca de 120 leituras, com intervalo de 5 s).
3. Relação entre a abertura da tampa e a subida de temperatura nos minutos seguintes.
4. Eficiência da ventoinha, medida pelo tempo que ela leva para baixar 1 °C.
5. Relatório diário com mínima, média e máxima, percentual do tempo dentro da faixa de 18 a 27 °C e número de eventos de atenção e crítico.

Os indicadores 1 e 2 fazem parte da camada de reforço definida na Etapa 1. Os demais entram se houver tempo. Na Etapa 5, antes dos testes na bancada, o cálculo será testado com dados simulados, alterando a temperatura do sensor no Wokwi. O módulo de cálculo terá testes automatizados com dados conhecidos.

11. Decisões e alterações em relação à proposta

Item	Proposta da Etapa 1	Definição da Etapa 2	Motivo
Broker MQTT	Na nuvem, serviço não definido	Público na nuvem (HiveMQ, com EMQX como alternativa)	O Wokwi não alcança um broker no notebook. O mesmo broker serve ao simulador e à bancada.
Erro de leitura	Não detalhado	Estado de falha de sensor, com a ventoinha ligada por segurança	A Etapa 3 exige tratamento básico de situações de erro.
Pinos	Não definidos	Definidos na seção 4	Evitam os pinos de inicialização e o ADC2, que não funciona com Wi-Fi.
Modo manual	Alternar entre automático e manual	Em manual, a ventoinha segue o comando, exceto em crítico e falha, em que fica ligada. LED e buzzer seguem o estado.	Evita desligar o alarme ou a refrigeração por engano ou por comando indevido.
Mensagens	Mensagem JSON periódica	Tópicos e campos definidos na seção 7	Padroniza o que o ESP32 envia e recebe.
Estrutura do banco	SQLite, sem tabelas definidas	Tabelas `readings` e `events` (seção 8)	Separa as medições periódicas dos acontecimentos pontuais.
Alimentação do DHT11	Não detalhada	3V3, com 5V só se a leitura falhar	O sensor funcionou com 3,3 V num teste anterior no laboratório. Assim o sinal de dados nunca passa de 3,3 V.
Montagem	Protoboard do kit	ESP32 entre duas protoboards de 830 pontos do laboratório, como no circuito do Wokwi (Anexo A)	A placa ocupa quase toda a largura de uma protoboard e sobra uma fileira de furos de um lado só. Com duas, sobram furos dos dois lados.

12. Cronograma atualizado e riscos

Etapa	Detalhamento	Depende de
3	Firmware com estados, histerese e falha de sensor. Montagem componente por componente. Teste do transistor com a ventoinha isolado.	Peças recebidas: DS18B20, ventoinha, transistor e diodo. Sem elas, a etapa usa o DHT11 como sensor e um LED no lugar da ventoinha.
4	MQTT no HiveMQ, servidor Python com banco e painel web, comandos nos tópicos da seção 7 e teste de reconexão.	Wi-Fi do laboratório com saída para a porta 1883. Hotspot do celular como alternativa.
5	Indicadores 1 e 2, plano de testes (aquecimento do sensor, abertura da tampa e queda do Wi-Fi) e dossiê.	Banco SQLite com dados reais.

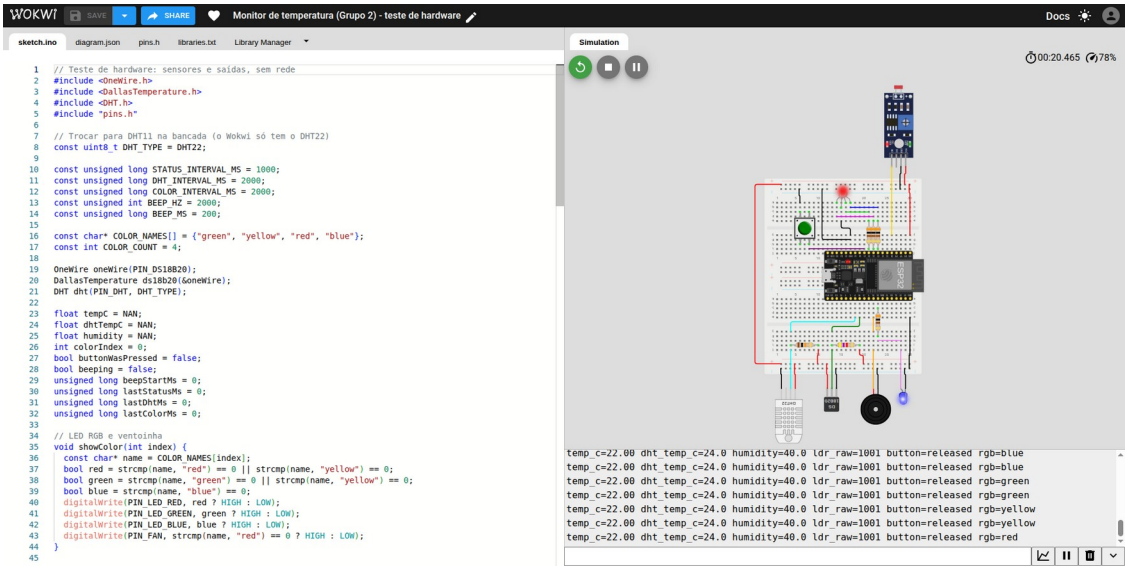
Riscos acrescentados nesta etapa:

Risco	Efeito	Resposta
Broker público sem autenticação	Terceiros podem ler ou publicar nos tópicos, inclusive comandos	Prefixo único da equipe, sem dados sensíveis, validação das mensagens, faixa permitida para os limites e ventoinha ligada em crítico e falha. Limitação registrada no dossiê.
Limiar do LDR muda com a iluminação do laboratório	Falsos alarmes de tampa aberta	Calibrar na bancada e deixar o limiar ajustável no código.
Corrente da ventoinha acima do esperado	O transistor esquenta	Conferir a corrente na etiqueta e usar um transistor de maior capacidade, se necessário.
O ESP32 reinicia quando a ventoinha liga	Perda de dados e funcionamento instável	Capacitor de 100 µF ou alimentação separada, com GND comum.
O servidor grava e lê o SQLite ao mesmo tempo (MQTT e API)	Erro de gravação ou de leitura	Abrir uma conexão por operação, com transações curtas. Testar com mensagens chegando enquanto o painel consulta.

13. Registro de evidências e anexos

Requisito ou funcionalidade	Tipo de evidência	Arquivo ou link	Como a evidência comprova
Diagrama de blocos	Diagrama	Figura 1	Mostra entradas, ESP32, saídas, broker e servidor Python.
Ligações e pinos	Diagrama e tabela	Figura 2 e seção 4	Relaciona cada componente ao seu GPIO.
Fluxo lógico	Diagrama de estados	Figura 3	Define estados, limites e histerese.
Mensagens e comandos	Tabelas e exemplo JSON	Seção 7	Define tópicos, campos e comandos.
Estrutura do banco	Tabelas	Seção 8	Define as tabelas e as colunas dos dados guardados.
Circuito simulado	Print e link do Wokwi	Anexo A, link do Wokwi e print no OneDrive	Mostra o circuito montado e funcionando no simulador.

Anexo A. Circuito no Wokwi.



Link do projeto: https://wokwi.com/projects/479892496574303233. O print mostra o sketch de teste de hardware rodando no simulador. A ESP32 fica entre duas protoboards, do jeito que será montada na bancada. Os sensores, o buzzer e o LED que faz o papel da ventoinha ficam fora delas, ligados por fios. Na captura, o LED RGB está vermelho, o LED da ventoinha está aceso e o monitor serial mostra as leituras do DS18B20, do DHT22 e do LDR. Como o simulador não tem DHT11, o circuito usa o DHT22. Print no OneDrive: https://catolicasc-my.sharepoint.com/:i:/g/personal/vinicius06_oliveira_catolicasc_edu_br/IQDQ9YldnoY0SofpAs_nNc_LASN8Mbad7wypbQCwvmmwtOY?e=keDGc6.